UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants. Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 10 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2015-06, page [2]

Unofficial model answer:

Von Neumann architecture consists of the CPU (control unit and ALU), memory, input devices, output devices, and the buses that interconnect them. One appropriate example for each component is: (1) Control Unit — the program counter (PC) or instruction register (IR), which fetches and decodes instructions; (2) Arithmetic Logic Unit (ALU) — the accumulator or an integer adder that performs arithmetic/logic operations; (3) Memory — RAM (main memory) that stores both instructions and data; (4) Input device — a keyboard that supplies data/instructions to the system; (5) Output device — a monitor/display that presents results; (6) Bus — the system bus (address bus, data bus, control bus) that carries addresses, data, and control signals between components.

Marking points:

- *1 mark: Control Unit example (e.g., program counter/instruction register)*
- *1 mark: ALU example (e.g., accumulator/adder)*
- *1 mark: Memory example (e.g., RAM/main memory)*
- *1 mark: Input device example (e.g., keyboard)*
- *1 mark: Output device example (e.g., monitor/display) OR bus example (system bus)*

Model confidence: medium.

Model note: The question asks for ONE example per component; the exact set of components expected is not specified in the extracted text. The answer lists the standard Von Neumann components (CU, ALU, memory, input, output, bus) and gives one example each. If the official scheme expects only five components (CU, ALU, memory, input, output), the bus example is extra; if it expects the bus as a component, the output example may be extra. Marking allocates 5 marks across five component examples.

QUESTION 2 (5 marks)

Source: CSC429-2026-02, page [2]

Unofficial model answer:

a) The integrated circuit (IC) is credited to Jack Kilby (Texas Instruments, 1958), who demonstrated the first working IC, with Robert Noyce (Fairchild, 1959) independently developing a practical silicon version; the usual exam answer is Jack Kilby.

b) The integrated circuit was first introduced in the third generation of computers (1964–1971), replacing discrete transistors and ushering in machines such as the IBM System/360.

c) Three main components of an integrated circuit are: (1) the active semiconductor devices — transistors (and diodes) formed on the silicon die; (2) passive components — resistors and capacitors (and their interconnections); and (3) the interconnection wiring/metallization and the semiconductor substrate (chip) on which they are fabricated, together with the package and bonding pads that connect the die to external pins.

Marking points:

- *1 mark (a): Jack Kilby (accept Robert Noyce as co-inventor; do not accept unrelated names).*
- *1 mark (b): Third generation.*
- *3 marks (c): 1 mark each for any three valid IC components, e.g. transistors, diodes, resistors, capacitors, interconnection wiring/metallization, semiconductor substrate/die, package/bonding pads.*

Model confidence: high.

Model note: No figure or table was supplied; the answer is based on standard CSC429 computer-generation material. Some textbooks credit Noyce alongside Kilby, so either name should be accepted for (a).

© Hak Cipta Universiti Teknologi MARA

QUESTION 3 (5 marks)

Source: CSC429-2023-02, page [3]

Unofficial model answer:

The two bytes are 1101 1100 and 0010 0111. In sign-and-magnitude representation, the most significant bit (bit 7) is the sign bit (0 = positive, 1 = negative) and the remaining seven bits are the magnitude. Thus 1101 1100 represents -($101\ 1100_2$) = -(64+16+8+4) = -92, and 0010 0111 represents +($010\ 0111_2$) = +(32+4+2+1) = +39. Adding the signed values gives -92 + 39 = -53. Therefore the decimal value of the sum is -53. Workflow: (1) identify sign and magnitude bits for each byte; (2) convert each magnitude to decimal; (3) apply the sign; (4) add the two signed decimal values. Note: if the sum were to be stored back in sign-and-magnitude, -53 would be 1011 0101, but the question only asks for the decimal value of the sum.

Marking points:

- *1 mark: correctly identifies sign bit and magnitude bits for 1101 1100 (sign 1, magnitude 101 1100).*
- *1 mark: correctly converts 1101 1100 to -92.*
- *1 mark: correctly converts 0010 0111 to +39.*
- *1 mark: correctly adds -92 + 39 = -53.*
- *1 mark: clear workflow showing sign/magnitude separation and decimal conversion steps.*

Model confidence: high.

Model note: Assumes standard 8-bit sign-and-magnitude with MSB as sign bit. No figure or table supplied; answer based on question text only. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 4 (5 marks)

Source: CSC429-2026-02, page [2]

Unofficial model answer:

a) The computer interface during the third generation (mid-1960s to early 1970s) was primarily based on interactive terminals connected to mainframe computers. These terminals included teletypewriters and CRT-based dumb terminals, which allowed users to interact with the system via command-line interfaces (CLI). The interface was character-oriented and text-based, with users typing commands and receiving textual output. In the fourth generation (early 1970s to present), the interface evolved significantly with the advent of microprocessors and personal computers. This generation saw the introduction of graphical user interfaces (GUIs), starting with Xerox PARC's Alto and later commercialized by Apple Macintosh and Microsoft Windows. GUIs use windows, icons, menus, and pointing devices (WIMP) to provide a more intuitive and user-friendly interaction. Additionally, the fourth generation introduced network interfaces, allowing computers to connect to local area networks and the Internet, and later touch-based and voice-based interfaces. Overall, the third generation was characterized by text-based terminal interfaces, while the fourth generation introduced graphical, networked, and multimodal interfaces.

b) In my opinion, the leading cause that changed the computer interface from the third to the fourth generation was the invention and widespread adoption of the microprocessor. The microprocessor enabled the development of affordable personal computers, which shifted computing from centralized mainframes to individual use. This democratization of computing created a need for interfaces that were accessible to non-expert users, driving the development of graphical user interfaces. Additionally, the microprocessor's increased processing power and decreasing cost made it feasible to implement complex graphical displays and pointing devices. Furthermore, the rise of software applications for productivity, entertainment, and communication demanded more intuitive interaction methods. Thus, the microprocessor revolution, by enabling personal computing and fostering user-centric design, was the primary catalyst for the interface change.

Marking points:

- *1 mark: Third generation interface described as text-based terminal/command-line (e.g., teletype, CRT dumb terminal).*

© Hak Cipta Universiti Teknologi MARA

• 1 mark: Fourth generation interface described as graphical user interface (GUI) with WIMP elements (windows, icons, menus, pointer).

• 1 mark: Additional fourth-generation interface features mentioned (e.g., network interface, touch/voice, multimedia).

• 1 mark: Leading cause identified as microprocessor invention/adoption enabling personal computers.

• 1 mark: Explanation linking cause to interface change (e.g., need for user-friendly interface for non-experts, increased processing power for graphics).

Model confidence: high.

Model note: The question is open-ended; answers may vary but should cover the key interface types and a plausible cause. The marking points are indicative and not an official scheme.

QUESTION 5 (10 marks)

Source: CSC429-2019-06, page [4, 5]

Unofficial model answer:

a) Access methods: (i) Cassette tape — sequential access (data read in order along the tape; to reach a later track the tape must be wound forward). (ii) Floppy disk — direct (random) access: the read/write head moves to the required track and the disk rotates to the sector, so blocks can be reached without reading everything before them. (iii) Cache memory — random access: any cache line is addressed directly by its tag/index and accessed in essentially constant time. (iv) RAM (main memory) — random access: any word is addressed directly via its address, independent of location. Note: the question's sub-labels (i)–(iv) are not visible in the extracted text; the mapping above follows the order of the four scenarios (tape, floppy, cache, RAM).

b) SRAM (Static RAM): built from flip-flop (typically 6-transistor) cells that hold a bit as long as power is supplied; no refresh needed; very fast (access times of a few nanoseconds); low density and high cost per bit, so used for cache. DRAM (Dynamic RAM): stores each bit as charge on a capacitor (1 transistor + 1 capacitor per cell); charge leaks, so it must be periodically refreshed (every few milliseconds); slower than SRAM but denser and cheaper per bit, so used for main memory.

c) In a three-level cache (L1, L2, L3), L1 is the fastest — it is smallest and closest to the CPU core, with the lowest access latency. Cache is implemented using SRAM, because SRAM's speed and lack of refresh make it suitable for the small, fast cache arrays.

Marking points:

• 1 mark: (i) cassette tape identified as sequential access

• 1 mark: (ii) floppy disk identified as direct/random access

• 1 mark: (iii) cache identified as random access

• 1 mark: (iv) RAM identified as random access

• 1 mark: SRAM described as flip-flop based, no refresh, fast, low density/high cost (cache use)

• 1 mark: DRAM described as capacitor-based, needs refresh, slower, high density/low cost (main memory use)

• 1 mark: SRAM vs DRAM comparison (speed/cost/density/refresh) clearly stated

• 1 mark: L1 identified as the fastest cache level

• 1 mark: SRAM identified as the RAM type used for cache

• 1 mark: justification linking L1 closeness/small size to lowest latency and/or SRAM speed to cache use

Model confidence: medium.

Model note: The sub-labels (i)–(iv) in part (a) are not present in the extracted question text; the mapping assumes the order of the four scenarios (tape, floppy, cache, RAM). This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 6 (10 marks)

Source: CSC429-2023-07, page [5]

Unofficial model answer:

a) A symmetric multiprocessor (SMP) organization redrawn from Figure 2: replace the distributed main

© Hak Cipta Universiti Teknologi MARA

memories (Memory 1, Memory 2) and the NUMA interconnect with a single shared main memory accessed uniformly by all processors. All processors connect to the same shared main memory through a common bus or interconnection network, and each processor typically has its own local cache. Unlike Figure 2, there is no memory that is 'closer' to one processor than another; every processor has equal access latency to the single shared memory. A typical redraw shows: Processor 1 and Processor 2 (each with local cache) connected via a shared system bus/interconnect to one Main Memory block, with all processors symmetric (no master/slave, no private main memory).

b) Two examples of tightly coupled MIMD parallel processor architectures: (1) Symmetric Multiprocessor (SMP) — multiple identical processors share a single main memory and I/O, with uniform memory access; (2) Non-Uniform Memory Access (NUMA) multiprocessor — multiple processors with distributed shared memory where access time depends on memory location, but all memories form one shared address space. (Other acceptable examples: clustered multiprocessors with shared memory, or multicore processors with shared cache/memory.)

c) Two disadvantages of tightly coupled MIMD parallel processor architectures: (1) Limited scalability — because all processors contend for a single shared memory/bus, adding more processors increases contention and saturates the interconnect, so performance does not scale well beyond a modest number of processors. (2) Memory access bottleneck and coherence overhead — simultaneous access to shared data causes bus/memory conflicts, and cache-coherence protocols add communication and synchronization overhead, increasing latency and complexity. (Also acceptable: higher cost/complexity of shared hardware, and single point of failure in shared memory.)

Marking points:

- *a) 1 mark: single shared main memory shown (no distributed/private main memories)*
- *a) 1 mark: all processors connected symmetrically to the shared memory via a common bus/interconnect*
- *a) 1 mark: each processor shown with its own local cache (or equivalent) and uniform access to memory*
- *b) 1 mark: Symmetric Multiprocessor (SMP) identified as a tightly coupled MIMD example*
- *b) 1 mark: NUMA (or another valid tightly coupled MIMD, e.g., shared-memory cluster/multicore) identified as a second example*
- *c) 1 mark: first disadvantage (e.g., limited scalability due to shared memory/bus contention)*
- *c) 1 mark: explanation/elaboration of first disadvantage*
- *c) 1 mark: second disadvantage (e.g., memory access bottleneck / cache-coherence and synchronization overhead)*
- *c) 1 mark: explanation/elaboration of second disadvantage*
- *Total: 10 marks*

Model confidence: medium.

Model note: Figure 2 is only available as a caption/extracted text; the redraw in (a) is described textually because the actual diagram cannot be reproduced. The answer assumes the standard CSC429 definition of tightly coupled MIMD (shared memory, e.g., SMP and NUMA). Mark allocation is inferred, not official.

QUESTION 7 (10 marks)

Source: CSC429-2024-01, page [3]

Unofficial model answer:

a) The two processes in the machine cycle governed by main memory are the Fetch phase and the Memory (data transfer) phase. In the Fetch phase, main memory supplies the instruction word addressed by the Program Counter (PC) to the CPU; the memory is the source and the instruction is placed on the data bus into the Instruction Register (IR). In the Memory phase (also called the memory read/write or data transfer phase), main memory acts as the source or destination for operands: a Load/Read transfers a data word from memory to a CPU register, while a Store/Write transfers a register value to memory. Both processes are 'governed' by main memory because the memory address register (MAR) selects the location and the memory control

© Hak Cipta Universiti Teknologi MARA

signals (Read/Write) determine the direction of transfer; the CPU initiates but the timing and completion depend on memory access.

b) Two operations comprised in the Execute phase are: (i) arithmetic/logic operations performed by the ALU (e.g., ADD, SUB, AND, OR, compare) on operands taken from registers or memory; and (ii) data transfer operations such as loading a register from memory or storing a register to memory (and register-to-register moves). Other acceptable examples include incrementing the PC, setting condition flags, or a branch/jump that updates the PC. The Execute phase decodes and carries out the operation specified by the instruction fetched in the previous phase.

c) During the Fetch cycle, the Program Counter (PC) holds the address of the next instruction to be fetched. Its content is copied to the Memory Address Register (MAR) so memory can be addressed; after the instruction is fetched, the PC is incremented (PC <- PC + 1, or by the instruction length) so it points to the following instruction. The Instruction Register (IR) receives and holds the fetched instruction word from main memory via the Memory Data Register (MDR)/data bus. The IR then supplies the instruction to the control unit and decoder, which interprets the opcode and generates the control signals for the Execute phase. Thus the PC is the address pointer that advances, while the IR is the holding register for the current instruction.

Marking points:

• *a) 1 mark: identifies Fetch phase as memory-governed (instruction read from memory using PC/MAR).*

• *a) 1 mark: identifies Memory/data-transfer phase as memory-governed (operand read/write between CPU and memory).*

• *a) 1 mark: explains role of MAR/address and Read/Write control in selecting memory location and transfer direction.*

• *b) 1 mark: names one valid Execute-phase operation (e.g., ALU arithmetic/logic operation).*

• *b) 1 mark: names a second valid Execute-phase operation (e.g., load/store data transfer or branch/PC update).*

• *b) 1 mark: briefly explains that Execute carries out the decoded operation.*

• *c) 1 mark: states PC holds address of next instruction and is copied to MAR.*

• *c) 1 mark: states PC is incremented after fetch to point to next instruction.*

• *c) 1 mark: states IR receives/holds the fetched instruction from memory.*

• *c) 1 mark: states IR feeds the control unit/decoder to interpret opcode and generate control signals.*

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure/table was supplied beyond the question text, so the answer is based on standard von Neumann machine-cycle descriptions. Mark allocation is indicative; the actual scheme may distribute the 10 marks differently.

QUESTION 8 (10 marks)

Source: CSC429-2018-12, page [4]

Unofficial model answer:

The CPU registers are small, high-speed storage locations inside the processor that hold the operands, results, addresses and control information needed during instruction execution. Their specific functions are: (1) Program Counter (PC) / Instruction Pointer: holds the address of the next instruction to be fetched; it is incremented (or loaded with a branch target) after each fetch so that the instruction sequence is maintained. (2) Instruction Register (IR): holds the instruction currently being fetched/decoded/executed; its opcode field feeds the control unit and its operand fields feed address generation. (3) Memory Address Register (MAR): holds the address of the memory location (or I/O port) to be accessed by the next memory read or write; it drives the address bus. (4) Memory Data Register (MDR) / Memory Buffer Register (MBR): holds the data word read from memory or the data word to be written to memory; it is the interface between the CPU and the data bus. (5) Accumulator (AC): a general-purpose working register that holds one operand before an ALU operation and receives the ALU result, so it is central to arithmetic/logic instruction execution. (6) General-Purpose Registers (GPRs): a set of programmer-visible registers used to hold operands, intermediate results,

© Hak Cipta Universiti Teknologi MARA

addresses and index values, reducing memory traffic; they may also serve as index or base registers for addressing. (7) Status/Flag Register (Program Status Word, PSW): holds condition flags (zero, sign, carry, overflow) set by the ALU and control bits (interrupt enable, mode bits) that affect subsequent instruction execution and branching. (8) Stack Pointer (SP): holds the address of the top of the stack in memory, used for push/pop, subroutine call/return and interrupt handling. (9) Temporary registers (e.g., A, B, ALU input/output latches): hold intermediate operands and results internal to the datapath during multi-step operations. (10) Index/Base registers (where present): hold offsets or base addresses used in indexed and base-relative addressing. In summary, PC and IR control the instruction stream, MAR and MDR interface with memory, AC/GPRs/temporaries hold data, PSW holds status/control, and SP supports stack operations.

Marking points:

• 1 mark: Program Counter (PC) holds the address of the next instruction to be fetched and is updated after fetch/branch.

• 1 mark: Instruction Register (IR) holds the instruction currently being decoded/executed.

• 1 mark: Memory Address Register (MAR) holds the address of the memory location to be read or written (drives address bus).

• 1 mark: Memory Data Register (MDR/MBR) holds data read from or to be written to memory (data-bus interface).

• 1 mark: Accumulator (AC) holds an operand and receives ALU results for arithmetic/logic operations.

• 1 mark: General-purpose registers hold operands, intermediate results and addresses for programmer use.

• 1 mark: Status/Flag register (PSW) holds condition flags (zero, carry, sign, overflow) and control bits used for branching and mode control.

• 1 mark: Stack Pointer (SP) holds the top-of-stack address for push/pop, subroutine calls and interrupts.

• 1 mark: Temporary registers hold intermediate operands/results within the datapath during instruction execution.

• 1 mark: Correct overall grouping/role (instruction-stream control, memory interface, data storage, status/control) and clear terminology.

Model confidence: medium.

Model note: The question text does not specify a particular CPU or list of registers, so the answer covers the standard CSC429 register set (PC, IR, MAR, MDR, AC, GPRs, PSW, SP, temporaries). If the course's lecture notes use a different register set or naming (e.g., only PC, IR, MAR, MDR, AC), marks should be mapped to those registers. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 9 (10 marks)

Source: CSC429-2018-12, page [2]

Unofficial model answer:

a) ENIAC belongs to the first generation of computers (roughly 1940–1956), which was characterised by vacuum-tube technology, machine-language programming and very large physical size.

b) ENIAC was built by John Presper Eckert and John William Mauchly at the Moore School of Electrical Engineering, University of Pennsylvania (with U.S. Army funding; the statement's 'Mr. X and Mr. Y' refers to these two designers).

c) The main technology used in ENIAC was the vacuum tube (thermionic valve); it used about 18,000 vacuum tubes together with relays, resistors and capacitors for its logic and storage.

d) Possible solutions to the problems faced when using ENIAC: Produced a lot of heat – use forced-air cooling/fans, air-conditioned machine rooms, and later replace vacuum tubes with transistors (solid-state devices) which dissipate far less heat. Consume a lot of energy – use more efficient power supplies and cooling, power the machine from a dedicated supply, and again migrate to transistor-based circuitry with lower power consumption. Large in size – replace bulky vacuum tubes and discrete components with transistors and then integrated circuits, use modular/rack packaging and magnetic-core or semiconductor memory to shrink the machine. Slow processing – replace vacuum tubes with faster transistors/ICs, increase clock speed, use parallel processing and faster memory (e.g., core memory instead of delay lines). Limited programming

© Hak Cipta Universiti Teknologi MARA

capabilities – provide a stored-program design (von Neumann architecture) with memory holding both data and instructions, use assembly/high-level languages and compilers, and provide input/output devices and an operating system to ease program entry and execution.

Marking points:

• 1 mark (a): Correctly states first generation (vacuum-tube generation).

• 1 mark (b): Names both John Mauchly and J. Presper Eckert (or 'Eckert and Mauchly') as builders; accept University of Pennsylvania/Moore School context.

• 1 mark (c): Identifies vacuum tubes as the main technology.

• 1 mark (d): Gives a valid solution for heat (e.g., cooling/air-conditioning or transistors).

• 1 mark (d): Gives a valid solution for energy consumption (e.g., efficient power supply or transistor/IC replacement).

• 1 mark (d): Gives a valid solution for large size (e.g., transistors/ICs, modular packaging, smaller memory).

• 1 mark (d): Gives a valid solution for slow processing (e.g., faster switching devices, higher clock, parallelism).

• 1 mark (d): Gives a valid solution for limited programming capability (e.g., stored-program concept, high-level languages/compilers, better I/O).

• 1 mark: Overall coherence/terminology (uses correct computer-organization terms such as vacuum tube, stored program, transistor, IC).

• 1 mark: Completes all subparts (a–d) with no missing problem in the table.

Model confidence: high.

Model note: The question text contains typographical errors in the problem table (e.g., 'Consume a lot of ener', 'Lar e in size', 'Slow rocessin', 'Limited ro rammin ca abilities'); these have been interpreted as energy, large size, slow processing and limited programming capabilities. The statement's 'Mr. X and Mr. Y' is taken to refer to Eckert and Mauchly, the known ENIAC designers. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 10 (10 marks)

Source: CSC429-2019-06, page [3]

Unofficial model answer:

a) Convert -176.8125$_{10}$ to IEEE 754 single precision hexadecimal: The number is negative, so sign bit S = 1. Convert the magnitude to binary: 176 = 10110000$_2$ and 0.8125 = 0.1101$_2$, so 176.8125 = 10110000.1101$_2$. Normalize: 1.01100001101$_2$ × 2^7. The exponent field E = 7 + 127 = 134 = 10000110$_2$. The fraction field is the 23 bits after the leading 1: 01100001101000000000000. Grouping into 4-bit nibbles: 1 10000110 01100001101000000000000 → 1100 0011 0011 0000 1101 0000 0000 0000 = C330D000$_{16}$. Therefore -176.8125$_{10}$ = C330D000$_{16}$ in IEEE single precision. b) Convert 3F00C000$_{16}$ to decimal: Expand to binary: 0011 1111 0000 0000 1100 0000 0000 0000. Fields: S = 0 (positive), exponent field E = 01111110$_2$ = 126, fraction F = 00000001100000000000000$_2$. The stored exponent is not zero or all ones, so the value is normal: exponent = E - 127 = -1, significand = 1.F = 1.000000011$_2$. Thus value = 1.000000011$_2$ × 2^{-1} = 0.1000000011$_2$ = 0.5 + 2^{-9} = 0.5 + 0.001953125 = 0.501953125$_{10}$. Therefore 3F00C000$_{16}$ = 0.501953125$_{10}$.

Marking points:

• 1 mark: correct sign bit S = 1 for the negative number in part (a).

• 2 marks: correct binary conversion of 176.8125 to 10110000.1101$_2$ and correct normalization to 1.01100001101$_2$ × 2^7.

• 2 marks: correct biased exponent 134 = 10000110$_2$ and correct 23-bit fraction field 01100001101000000000000.

• 1 mark: correct final hexadecimal answer C330D000$_{16}$ for part (a).

• 1 mark: correct binary expansion of 3F00C000$_{16}$ and correct field extraction S=0, E=126, F=00000001100000000000000 in part (b).

• 2 marks: correct computation of exponent -1 and significand 1.000000011$_2$.

• 1 mark: correct final decimal value 0.501953125$_{10}$ for part (b).

Model confidence: high.

Model note: The question text contains a typo: '3FOOC00016' is interpreted as 3F00C000$_{16}$ (hexadecimal). No figure or table is

© Hak Cipta Universiti Teknologi MARA

supplied. This is an unofficial model answer, not an official UTM marking scheme.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2023-07, page [6]

Unofficial model answer:

Figure 3 represents the expression 242 − (B + AC), i.e. 242 minus the sum of register B and the product of registers A and C. Registers A, B and C are read-only; x, y and z are temporaries; the final result must end in register y. The three programs below use only the mnemonics listed in Table 1.

Zero-address program: With a stack machine, operands are pushed and the arithmetic instructions pop their operands and push the result. The constant 242 is pushed first, then B, then A and C are pushed and MUL is executed to leave AC on the stack; ADD then combines B and AC; SUB computes 242 − (B + AC); finally pop stores the top of stack into y. Program: PUSH 242; PUSH B; PUSH A; PUSH C; MUL; ADD; SUB; POP y. Comments: each PUSH places an operand on the stack; MUL replaces A and C by A*C; ADD replaces B and A*C by B+AC; SUB replaces 242 and (B+AC) by 242−(B+AC); POP y writes the result to y.

One-address program: A one-address machine uses an implicit accumulator. LOAD copies a memory/register operand into the accumulator; ADD, SUB, MUL and DIV combine the accumulator with the named operand and leave the result in the accumulator; STOR copies the accumulator to the named destination. Because A, B and C cannot be modified, the product AC is formed in a temporary: LOAD A; MUL C; STOR x (x = A*C); LOAD B; ADD x (accumulator = B + AC); STOR z (z = B + AC); LOAD 242; SUB z (accumulator = 242 − (B + AC)); STOR y. Comments: the first three instructions compute AC into x; the next two compute B + AC into z; the last three compute the final subtraction and store the answer in y.

Three-address program: Each three-address instruction has the form OP dest, src1, src2 and does not destroy its sources. Using temporaries x, y and z: MUL x, A, C (x = A*C); ADD z, B, x (z = B + AC); SUB y, 242, z (y = 242 − (B + AC)). Comments: MUL forms the product without changing A or C; ADD forms the parenthesised sum without changing B; SUB performs the final subtraction and places the required final answer directly in y. No STOR is needed because the destination of the last instruction is y.

Marking points:

• 2 marks: correctly identifies the expression as 242 − (B + AC) and respects the read-only constraint on A, B, C (uses only x, y, z as temporaries).

• 6 marks: zero-address program — correct use of PUSH/POP and stack order (2), correct MUL for AC (1), correct ADD for B+AC (1), correct SUB order 242 − (B+AC) (1), final POP y (1).

• 6 marks: one-address program — correct LOAD/STOR/accumulator usage (2), computes AC into a temporary without modifying A or C (1), computes B+AC (1), computes 242 − (B+AC) in the right order (1), final STOR y (1).

• 4 marks: three-address program — MUL x, A, C (1), ADD z, B, x (1), SUB y, 242, z (1), final answer in y and no source registers modified (1).

• 2 marks: correct assembly notation and meaningful comments throughout (award proportionally across the three programs).

Model confidence: high.

Model note: Figure 3 is supplied only as extracted text '242 - (B + AC)'; the answer assumes this is the intended expression. Table 1 lists only mnemonics, not operand syntax, so standard zero-, one- and three-address operand forms are assumed. The one-address column in the extracted table omits MUL; the answer assumes MUL is available on the one-address machine because the question requires computing AC. This is an unofficial model answer, not an official UTM marking scheme.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA